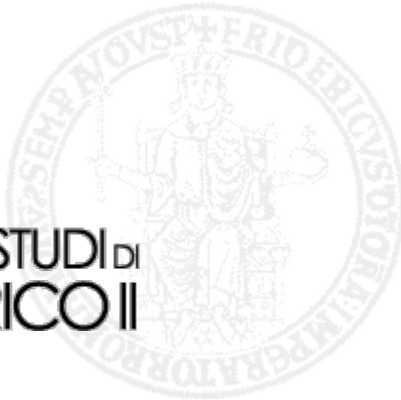




Cloud Platform And Infrastructure as Code

*Example of deploying an affordable
e-commerce platform: Scalamarket*

Indice

1	Presentazione del Progetto	2
1.1	Descrizione Generale	2
1.2	Requisiti dell'Applicativo	3
1.3	Il Sito Web	3
2	Orchestrazione Kubernetes	6
2.1	Infrastruttura del Cluster	6
2.1.1	Ridondanza e Scalabilità	6
2.1.2	Kubernetes Ingress Controller	7
2.1.3	Analisi dei Manifest Kubernetes	7
3	Deployment in AWS	10
3.1	Deployment tramite Infrastructure-as-Code (Terraform)	10
3.1.1	Manifest Ingress per AWS (ingress.yaml)	12
3.1.2	Load Balancing e Path-Based Routing	12
3.1.3	Configurazione delle credenziali IAM per Terraform	12
3.1.4	Selezione dello Use Case per l'Access Key	13
3.1.5	Accesso Remoto e Deploy Applicativo	14
4	Proposte di Integrazione e Sviluppo Futuro	18
4.1	Integrazioni Edge Computing e Scaling On-Premise	18
4.2	Benchmark Prestazionali: Cloud vs Locale	19
5	Documentazione Endpoints API	21
6	Interfaccia Utente (UI/UX)	26
6.1	Design e Suddivisione dei Ruoli	26
6.2	Pannello Cliente	28
6.3	Pannelli di Amministrazione	29
7	Guide al Deployment	32
7.1	Deployment in Ambiente Locale	32
7.2	Deployment in AWS tramite Infrastructure-as-Code (Terraform)	33

Abstract

Il presente documento illustra la progettazione, lo sviluppo e il *deployment* di **Scalamarket**, una piattaforma e-commerce basata su una moderna applicazione web scalabile a seconda delle esigenze di clientela, traffico web, magazzino, personale e via scorrendo. L'obiettivo del progetto è di implementare un'architettura a microservizi sufficiente per una piccola attività commerciale, che può anche espandersi con più sedi, e come questo possa essere effettuato gradualmente. In una prima parte ci si focalizzerà sulle componenti dell'ecosistema, formato da Frontend, Auth Service, Inventory Service, Order Service, interfacciati con database PostgreSQL, gestiti mediante orchestrazione **Kubernetes (K3s)** in un implementazione locale. Viene inoltre analizzato il processo di *provisioning* dell'infrastruttura cloud tramite **Terraform** su **Amazon Web Services (AWS)**, con particolare attenzione alle dinamiche di bilanciamento del carico (*Application Load Balancer*) e di instradamento (*Ingress Controller*), impostando anche dei limiti di budget. La relazione documenta l'evoluzione architetturale, partendo da un'impostazione locale essenziale, fino ad arrivare a una topologia orientata all'alta affidabilità (*High Availability*), concludendosi le rotte API utilizzate e come il progetto può essere portato avanti in futuro.



Figura 1: Logo di Scalamarket, ispirato principalmente da un minimarket di proprietà della famiglia di un amico



Capitolo 1

Presentazione del Progetto

1.1 Descrizione Generale

Scalamarket è una moderna piattaforma e-commerce basata su un'architettura a microservizi, progettata per garantire elevata scalabilità, affidabilità e flessibilità. Il sistema gestisce l'intero ciclo di vita dell'acquisto online: dalla registrazione degli utenti, alla consultazione del catalogo, fino all'elaborazione degli ordini e alla gestione logistica delle spedizioni. L'intera infrastruttura è stata ideata per essere distribuita in modo efficiente tramite Kubernetes ed eventualmente ospitata in ambiente cloud AWS, fornendo un backend solido, scalabile e supportato da API RESTful.

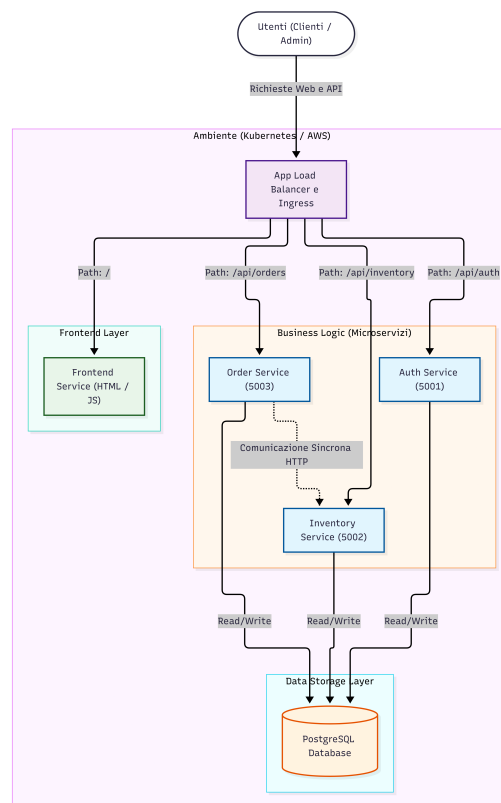


Figura 1.1: Diagramma dei Componenti UML



1.2 Requisiti dell'Applicativo

L'applicativo è stato sviluppato per soddisfare i seguenti requisiti architetturali, funzionali e di business

- **Autenticazione e Autorizzazione (Auth Service):** Il sistema deve permettere la registrazione e il login sicuri sia per i clienti (Client) che per gli amministratori (Admin). L'amministratore deve avere i privilegi per gestire le utenze.
- **Gestione Inventario (Inventory Service):** Il sistema deve gestire un catalogo dinamico di articoli, tenendo traccia delle quantità ripartite in molteplici filiali. Deve inoltre impedire la creazione di ordini per articoli non disponibili.
- **Gestione Ordini e Logistica (Order Service):** L'applicativo deve processare il carrello dell'utente, allocando in tempo reale i prodotti necessari e scalandoli in modo intelligente dalle filiali che ne hanno disponibilità. Deve inoltre tracciare l'occupazione dei mezzi logistici, gestendo i furgoni in uso e quelli a disposizione.
- **Architettura a Microservizi e Dati Isolati:** Per massimizzare la modularità, ogni servizio deve possedere una propria logica di business e interfacciarsi con il proprio database PostgreSQL indipendente, disaccoppiando così i domini applicativi.

È importante specificare che l'applicativo è da considerarsi un Prototipo (Proof of Concept) atto a dimostrare le dinamiche di scalabilità, distribuzione e interazione tra microservizi, e non è pensato per un utilizzo commerciale in produzione. Alcune interfacce, così come la maggior parte delle funzionalità di sicurezza sono da considerarsi basilari

1.3 Il Sito Web

Per l'implementazione di Scalamarket si è scelto un framework Flask. Il lato client e il pannello di gestione amministrativa sono stati realizzati mediante un'interfaccia web responsive integrata con il backend. L'applicazione espone e gestisce tutti gli endpoint necessari, mentre la UI comunica costantemente con questi servizi tramite chiamate asincrone.

Il sito implementa tre diverse interfacce: **Cliente**, **Gestore Filiale** ed **Amministratore Globale**.



Example of deploying an affordable e-commerce platform: Scalamarket

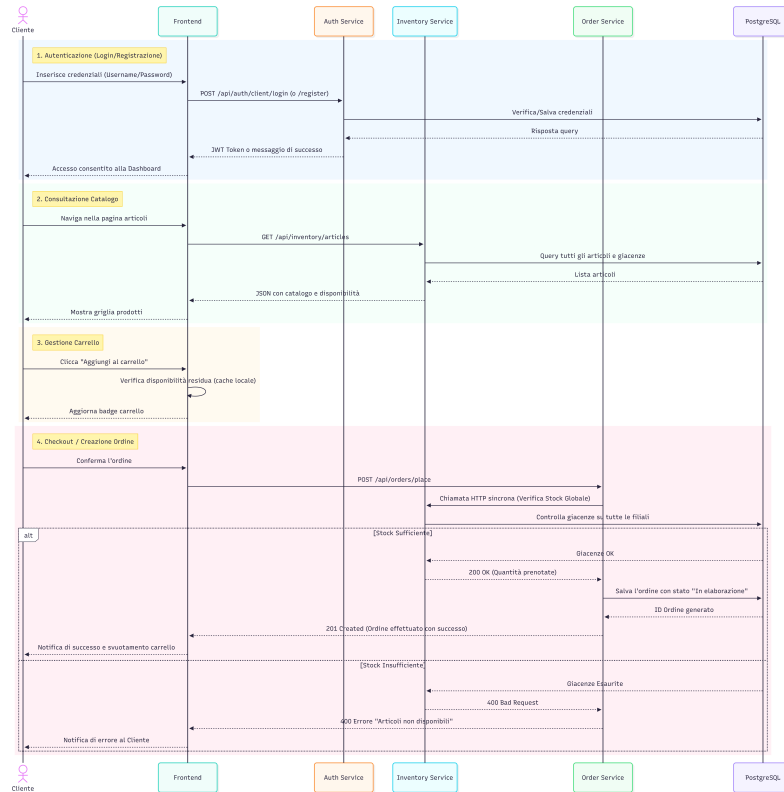


Figura 1.2: Diagramma di Sequenza

- **Interfaccia Cliente:** L'utente può scorrere l'intero catalogo, visualizzare i prezzi e le disponibilità massime, aggiungere elementi al carrello e, una volta inviato l'ordine, monitorarne lo stato e i dettagli dei prodotti tramite un'apposita cronologia.
- **Interfaccia Amministratore Globale:** L'admin ha accesso a una dashboard completa che gli permette di visualizzare l'inventario globale (con possibilità di ricerca rapida), controllare in tempo reale lo stato logistico totale, visionare ed eliminare lo storico ordini globale e monitorare/eliminare gli account utenti.
- **Interfaccia Gestore Filiale:** Il gestore ha una visione limitata esclusivamente alle giacenze e ai veicoli della propria sede, con i poteri esclusivi di poter aggiungere o rimuovere articoli dal catalogo, e aggiornare le scorte e i mezzi assegnati alla filiale.

Nota: per un'analisi dettagliata e visiva di come è strutturata l'interfaccia utente (UI) nelle sue varie schermate, si rimanda al quinto capitolo, "Interfaccia Utente", del documento.



Example of deploying an affordable e-commerce platform: Scalamarket

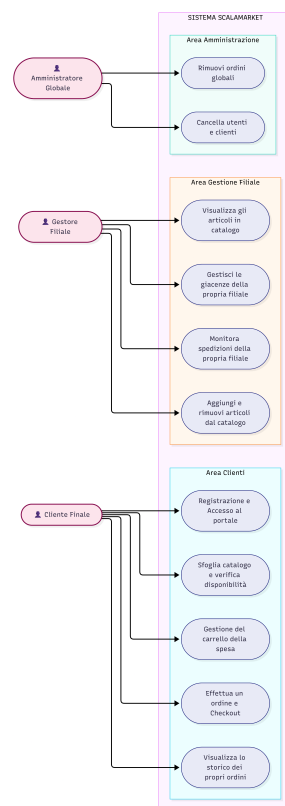


Figura 1.3: Diagramma dei Casi d'Uso



Capitolo 2

Orchestrazione Kubernetes

2.1 Infrastruttura del Cluster

L'infrastruttura Kubernetes poggia su componenti isolati al fine di scalare singolarmente in base alle esigenze e prevenire disservizi totali (*Single Point of Failure*).

I componenti base dell'applicativo sono i seguenti:

- **frontend** (nginx): Gestisce la GUI per clienti e amministratori, esposto sulla porta 8080.
- **auth-service** (Flask): Gestisce autenticazione, registrazione e ruoli.
- **inventory-service** (Flask): Gestisce il catalogo, articoli e giacenze.
- **order-service** (Flask): Gestisce gli ordini, lo storico e la logistica.
- **postgres**: Database centrale con storage persistente garantito da un `PersistentVolume` (PV) e un `PersistentVolumeClaim` (PVC), logicamente frammentato in 3 database separati.

2.1.1 Ridondanza e Scalabilità

I servizi stateless (Frontend e Logica di Business) sono stati configurati con repliche multiple, distribuite automaticamente dal `Service ClusterIP` che funge da load-balancer interno:

- **Frontend (nginx)**: 2 repliche per garantire alta disponibilità durante aggiornamenti o guasti di un Pod.
- **Logica di Business (auth, inventory, order)**: 3 repliche ciascuno, in quanto soggetti a maggior stress e richieste concorrenti (es. elaborazione parallela ordini o scritture simultanee inventario).
- **Database (postgres)**: Rimane a 1 replica. In ambiente nativo relazionale, la coerenza dei dati orizzontale richiede approcci dedicati; mantenere 1 replica ne previene il partizionamento anomalo.



Tutti i container Flask sono inoltre dotati di *resources* (requests e limits) per evitare che un consumo anomalo da parte di un Pod degradi le prestazioni degli altri (*noisy neighbor*).

2.1.2 Kubernetes Ingress Controller

L'accesso esterno e il routing dei servizi avvengono tramite un **Kubernetes Ingress Controller** (basato su NGINX) che agisce da *reverse proxy* HTTP a livello di cluster. Attraverso un unico indirizzo esposto al pubblico, elimina la necessità di port-forward multipli. Le rotte principali implementate sono:

- `/` → `frontend-service:8080`
- `/api/auth/` → `auth-service:5001`
- `/api/inventory/` → `inventory-service:5002`
- `/api/orders/` → `order-service:5003`

Nota: Si rimanda al quarto capitolo per un'analisi più dettagliata delle rotte implementate e quali richieste vengono gestite da ciascuna di esse.

2.1.3 Analisi dei Manifest Kubernetes

L'architettura descritta è governata da una serie di manifest Kubernetes (file YAML) che dichiarano in maniera esplicita lo stato desiderato dell'infrastruttura. Di seguito l'analisi dettagliata dei file che compongono il deployment:

- **ConfigMap e Storage (`postgres-configmap.yaml`, `postgres-pv-pvc.yaml`):** Il primo fornisce al database gli script SQL necessari per l'inizializzazione automatica delle tabelle al primo avvio. Il secondo definisce le risorse di archiviazione persistente (Persistent Volume Claim), essenziali per evitare la perdita di dati al riavvio del container PostgreSQL.
- **Database (`postgres-deployment.yaml`, `postgres-service.yaml`):** Il deployment gestisce l'istanza PostgreSQL collegandovi il volume persistente e il ConfigMap. Il Service espone internamente il database sulla porta 5432, permettendo ai microservizi di collegarsi tramite il nome host del servizio stesso.
- **Servizi di Backend (`auth-*`, `inventory-*`, `order-*`):** Per ciascun microservizio sono definiti un *Deployment* e un *Service*. Il *Deployment* specifica l'immagine Docker da prelevare, il numero di



Example of deploying an affordable e-commerce platform: Scalamarket

repliche desiderate (garantendo l'alta affidabilità) e inietta le variabili d'ambiente necessarie (come le credenziali del database). Inoltre, impone vincoli rigidi sulle risorse hardware richieste e sui limiti massimi (CPU e RAM) per evitare colli di bottiglia. Il *Service* funge da bilanciatore di carico interno, esponendo i Pod sulla rete del cluster.

- **Interfaccia Utente (`frontend-deployment.yaml`, `frontend-service.yaml`):** Gestiscono l'applicazione web statica. Il deployment assicura che il server NGINX sia in esecuzione fornendo i file HTML/JS/CSS agli utenti, mentre il Service instrada le richieste HTTP in entrata verso i Pod del frontend.
- **Ingress Locale (`ingress-local.yaml`):** Agisce da punto di ingresso unico per le richieste provenienti dall'esterno del cluster. Si basa sul controller NGINX e si occupa di smistare il traffico. Ad esempio, le richieste destinate ai percorsi dell'inventario o degli ordini vengono incanalate verso i rispettivi servizi backend, mentre la radice viene diretta al Frontend, mascherando all'utente la complessità dell'infrastruttura sottostante.

Di seguito viene riportato l'output dell'ambiente locale in esecuzione, che conferma lo stato Running di tutti i pod e la corretta visualizzazione della piattaforma dal browser locale.

```
● mone@Trattore: /mnt/c/Users/SiMonee/Desktop/CPIAC_Scalamarket$ kubectl get ingress
NAME                                CLASS    HOSTS    ADDRESS    PORTS    AGE
scalamarket-ingress-local          nginx    *        172.19.0.4  80      116s
● mone@Trattore: /mnt/c/Users/SiMonee/Desktop/CPIAC_Scalamarket$ kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
auth-service-6b5f789498-s5mzn      1/1     Running   2 (6m5s ago)  6m24s
frontend-cf799b88-hlt5f            1/1     Running   0              6m24s
inventory-service-66d9498f5-mbdpb  1/1     Running   2 (6m3s ago)  6m24s
order-service-76cfb6758-9rfhs      1/1     Running   2 (5m54s ago) 6m24s
postgres-7475d95dcf-ntcnm         1/1     Running   0              6m23s
○ mone@Trattore: /mnt/c/Users/SiMonee/Desktop/CPIAC_Scalamarket$
```

Figura 2.1: Output del comando `kubectl get pods` nell'ambiente locale



Example of deploying an affordable e-commerce platform: Scalamarket

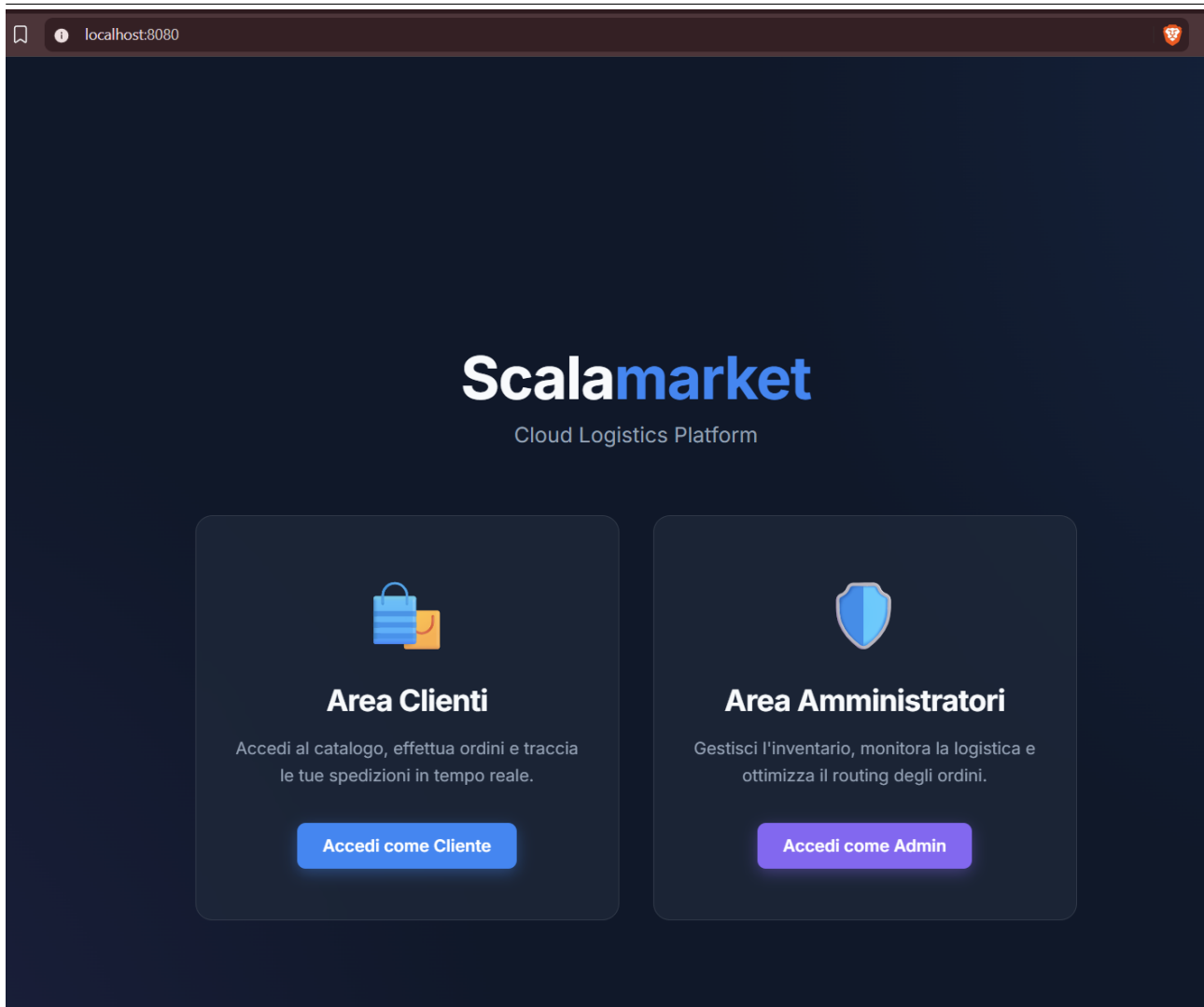


Figura 2.2: Piattaforma correttamente avviata ed esposta tramite localhost



Capitolo 3

Deployment in AWS

3.1 Deployment tramite Infrastructure-as-Code (Terraform)

Al fine di automatizzare e rendere riproducibile l'infrastruttura Cloud, è stato creato uno script Terraform per effettuare il provisioning delle risorse su Amazon Web Services (AWS). Questa metodologia, nota come *Infrastructure-as-Code* (IaC), garantisce che la configurazione del cloud sia scritta in file dichiarativi.

I file Terraform implementati (`main.tf`, `variables.tf`, `outputs.tf`) descrivono la seguente architettura:

- **VPC e Subnet:** Creazione di una Virtual Private Cloud (VPC) con subnet pubbliche per bilanciatori e risorse accessibili, e subnet private per i nodi Kubernetes.
- **Internet Gateway e Route Tables:** Per consentire il routing verso l'esterno.
- **Security Group:** Regole firewall stateful per aprire le porte in ingresso al traffico HTTP/HTTPS e porte interne per la comunicazione intra-nodo.
- **Application Load Balancer (ALB):** Per distribuire il traffico degli utenti verso i worker node.
- **Istanze EC2:** Lancio di due macchine virtuali distinte (`t3.medium`) basate su Amazon Machine Image (AMI) Ubuntu: una dedicata alle applicazioni clienti (nella Subnet 1) e una alle applicazioni admin (nella Subnet 2). Lo script `user_data.sh` automatizza l'installazione di Docker e delle componenti di Kubernetes (K3s) all'avvio su entrambe.
- **AWS Budgets:** Per evitare costi imprevisti, è configurato un *Monthly Budget* con limite di spesa pari a 100 USD mensili, che invia alert via email se i costi superano la soglia dell'80%. L'utilizzo di un'architettura essenziale a due nodi garantisce resilienza pur contenendo i costi rispetto a un classico cluster multi-nodo pesante.



Example of deploying an affordable e-commerce platform: Scalamarket

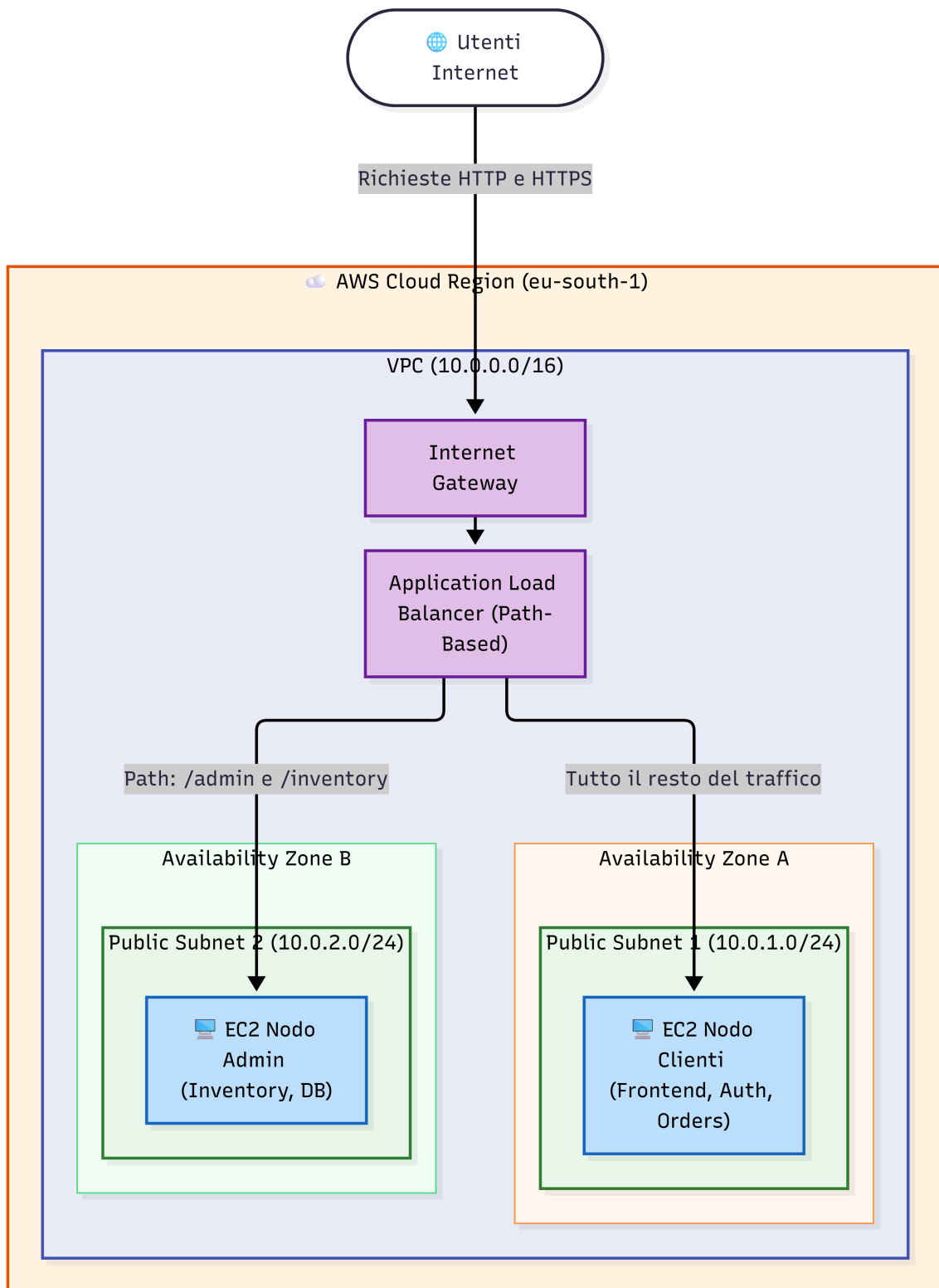


Figura 3.1: Schema Architetture AWS Multi-Nodo (VPC, Subnet, EC2, ALB)



3.1.1 Manifest Ingress per AWS (`ingress.yaml`)

A differenza del manifest locale (`ingress-local.yaml`) che si affida all'Ingress Controller NGINX, il file per l'infrastruttura AWS adotta traefik come *ingressClassName*, in quanto Kubernetes K3s ne include il supporto predefinito risultando ideale e "leggero" per l'integrazione con l'Application Load Balancer sovrastante. Il manifest AWS differisce in modo sostanziale dal locale anche perché frammentato: separa in definizioni YAML del tutto distinte l'Ingress per la rotta inventario, auth, frontend e orders. Questo asseconda l'architettura a nodi separati implementata su AWS, poiché il cluster non è unificato: delegando l'instradamento di più alto livello all'ALB di Amazon Cloud, si ottiene un instradamento corretto senza la necessità di DNS interni complessi tra i due nodi.

3.1.2 Load Balancing e Path-Based Routing

Al fine di separare le competenze mantenendo un singolo punto d'accesso, l'Application Load Balancer (ALB) è stato configurato per effettuare un *Path-Based Routing*. Il traffico in entrata viene analizzato dall'ALB: le richieste dirette ai path `/admin*` e `/inventory*` vengono inoltrate automaticamente al *Target Group* del nodo Admin (situato nella Subnet 2), mentre tutte le restanti richieste (di default) sono servite dal *Target Group* del nodo Clienti (nella Subnet 1).

3.1.3 Configurazione delle credenziali IAM per Terraform

Per consentire a Terraform di effettuare il provisioning delle risorse su AWS, è necessario configurare correttamente un utente programmatico tramite IAM (Identity and Access Management). Seguendo le best practice illustrate nei tutorial di riferimento (come l'aggiunta dell'utente a un Gruppo IAM specifico), si consiglia di applicare al gruppo la policy predefinita `AdministratorAccess`. Considerando che lo script Terraform necessita di orchestrare contemporaneamente VPC, Security Group, Istanze EC2, Application Load Balancer e impostazioni di Billing (per l'allarme budget), l'accesso amministratore garantisce un'esecuzione senza blocchi o errori di permessi per scopi didattici e di test. In scenari di produzione reali, tuttavia, andrebbe rigorosamente applicato il principio del privilegio minimo (Least Privilege), assemblando policy granulari (ad es. `AmazonEC2FullAccess`, `AmazonVPCFullAccess`, ecc.).



3.1.4 Selezione dello Use Case per l'Access Key

Durante la procedura guidata di generazione della coppia di chiavi di accesso (*Access Key* e *Secret Access Key*) nella AWS Console, viene richiesto di specificare l'ambito di utilizzo (*Use case*):

- L'opzione da selezionare è **Command Line Interface (CLI)**. Terraform, infatti, opera interfacciandosi con le API di AWS esattamente come un tool a riga di comando locale (o caricando le credenziali tramite l'AWS CLI configurata con `aws configure`, oppure leggendo i file `.aws/credentials` o variabili d'ambiente).
- Spuntando la casella di conferma relativa alle raccomandazioni di sicurezza AWS ("I understand the above recommendation and want to proceed..."), è possibile procedere con la generazione e il download del file `.csv` contenente le chiavi.

I passi per eseguire lo script sono i seguenti:

1. **Generazione Chiave SSH:** Terraform richiede l'esistenza di una chiave SSH locale per l'autenticazione remota sui server.
2. **Configurazione Credenziali AWS:** È necessario configurare la CLI con le *Access Keys* ottenute, operazione propedeutica all'esecuzione dello script.

```
mone@Trattore:/mnt/c/Users/SiMonee/Desktop/CPIAC_Scalamarket$ aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.

AWS Access Key ID [None]: 
AWS Secret Access Key [None]: 
Default region name [None]: eu-south-1
Default output format [None]: 
mone@Trattore:/mnt/c/Users/SiMonee/Desktop/CPIAC_Scalamarket$
```

Figura 3.2: Configurazione tramite AWS CLI

3. **Inizializzazione:** L'ambiente Terraform viene inizializzato scaricando i provider necessari per l'infrastruttura.

```
mone@Trattore:/mnt/c/Users/SiMonee/Desktop/CPIAC_Scalamarket/terraform$ terraform init
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.
mone@Trattore:/mnt/c/Users/SiMonee/Desktop/CPIAC_Scalamarket/terraform$
```

Figura 3.3: Inizializzazione di Terraform



Example of deploying an affordable e-commerce platform: Scalamarket

4. **Verifica e Applicazione:** Si esegue prima una verifica dei piani di esecuzione e infine si applicano le modifiche per effettuare il provisioning effettivo delle risorse su AWS.

```
none@frattore:/mnt/c/Users/SIUonee/Desktop/CPIAC_Scalamarket/terraform$ terraform apply
aws_lb.k8s_alb: Still creating... [00m30s elapsed]
aws_lb.k8s_alb: Still creating... [00m40s elapsed]
aws_lb.k8s_alb: Still creating... [00m50s elapsed]
aws_lb.k8s_alb: Still creating... [01m00s elapsed]
aws_lb.k8s_alb: Still creating... [01m10s elapsed]
aws_lb.k8s_alb: Still creating... [01m20s elapsed]
aws_lb.k8s_alb: Still creating... [01m30s elapsed]
aws_lb.k8s_alb: Still creating... [01m40s elapsed]
aws_lb.k8s_alb: Still creating... [01m50s elapsed]
aws_lb.k8s_alb: Still creating... [02m00s elapsed]
aws_lb.k8s_alb: Still creating... [02m10s elapsed]
aws_lb.k8s_alb: Still creating... [02m20s elapsed]
aws_lb.k8s_alb: Creation complete after 2m22s [id=arn:aws:elasticloadbalancing:eu-south-1:867637277337:loadbalancer/app/scalamarket-alb/0add823670c34ef1]
aws_lb_listener.front_end: Creation complete after 0s [id=arn:aws:elasticloadbalancing:eu-south-1:867637277337:listener/app/scalamarket-alb/0add823670c34ef1/64fefcf55a5a003cd]

Apply complete! Resources: 16 added, 0 changed, 0 destroyed.

Outputs:
alb_dns_name = "scalamarket-alb-171741115.eu-south-1.elb.amazonaws.com"
node_admin_public_ip = "15.160.113.28"
node_client_public_ip = "15.161.125.22"
none@frattore:/mnt/c/Users/SIUonee/Desktop/CPIAC_Scalamarket/terraform$
```

Figura 3.4: Output del comando Terraform Apply

3.1.5 Accesso Remoto e Deploy Applicativo

Al termine dell'esecuzione di Terraform, verranno stampati a video il DNS name dell'ALB e gli IP Pubblici delle due istanze EC2. L'infrastruttura a questo punto è in esecuzione ma vanno deployate le applicazioni.

- **Accesso SSH e Download Sorgenti:** Effettuato l'accesso alle macchine tramite le chiavi SSH, si esegue un clone parziale del repository scaricando solamente le cartelle necessarie (frontend, microservices e k8s).
- **Deploy K8s - Nodo Clienti:** Poiché i servizi sono distribuiti su nodi indipendenti, il servizio ordini viene dinamicamente riconfigurato per puntare all'indirizzo pubblico del Load Balancer. Vengono compilate le immagini Docker per i servizi di competenza del nodo (Frontend, Auth e Order), importate in K3s e vengono applicati i manifest per il database e per l'Ingress.



Example of deploying an affordable e-commerce platform: Scalamarket

```
ubuntu@ip-10-0-1-145:~$ sudo kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
auth-service-6b5f789498-2sbhw       1/1     Running   1 (49m ago) 55m
auth-service-6b5f789498-ct4v5       1/1     Running   0           55m
auth-service-6b5f789498-fshm9       1/1     Running   0           55m
frontend-57654766f5-6vqq6           1/1     Running   0           40m
frontend-57654766f5-g2v9c           1/1     Running   0           40m
inventory-service-66d9498f5-4z79k    0/1     ImagePullBackOff 0           76m
inventory-service-66d9498f5-fvv4p    0/1     ImagePullBackOff 0           76m
inventory-service-66d9498f5-jq5fk    0/1     ImagePullBackOff 0           76m
order-service-5bc9579bcc-4sg4x       1/1     Running   0           14m
order-service-5bc9579bcc-hwjyh       1/1     Running   0           14m
order-service-5bc9579bcc-kg7nk       1/1     Running   0           14m
postgres-7475d95dcf-phxlq           1/1     Running   0           76m
ubuntu@ip-10-0-1-145:~$ sudo kubectl get ingress
NAME                CLASS    HOSTS      ADDRESS      PORTS    AGE
auth-ingress        traefik  *          10.0.1.145   80       60m
frontend-ingress    traefik  *          10.0.1.145   80       60m
inventory-ingress    traefik  *          10.0.1.145   80       60m
order-ingress        traefik  *          10.0.1.145   80       60m
scalamarket-ingress nginx     *          10.0.1.145   80       76m
ubuntu@ip-10-0-1-145:~$
```

Figura 3.5: Verifica dello stato dei Pod sul Nodo Clienti AWS

- **Deploy K8s - Nodo Admin:** Sul nodo Admin si compila l'immagine per il servizio `inventory`, la si importa nell'ambiente K3s e si effettua il deploy delle risorse Kubernetes relative al database, all'inventario e all'Ingress dedicato.



Example of deploying an affordable e-commerce platform: Scalamarket

```
ubuntu@ip-10-0-2-77:~$ sudo kubectl get ingress
NAME                CLASS    HOSTS    ADDRESS    PORTS    AGE
auth-ingress        traefik  *        10.0.2.77  80       52m
frontend-ingress    traefik  *        10.0.2.77  80       52m
inventory-ingress    traefik  *        10.0.2.77  80       52m
order-ingress        traefik  *        10.0.2.77  80       52m
scalamarket-ingress  nginx    *        10.0.2.77  80       75m
ubuntu@ip-10-0-2-77:~$ sudo kubectl get pods
NAME                                READY    STATUS              RESTARTS    AGE
auth-service-6b5f789498-75skv      0/1      ImagePullBackOff    0           75m
auth-service-6b5f789498-nz429      0/1      ImagePullBackOff    0           75m
auth-service-6b5f789498-sfcfh      0/1      ImagePullBackOff    0           75m
frontend-58c88cff98-lb28h          1/1      Running              0           43m
frontend-58c88cff98-zrhgr          1/1      Running              0           43m
inventory-service-66d9498f5-42rv2   1/1      Running              0           53m
inventory-service-66d9498f5-9hmfk   1/1      Running              0           53m
inventory-service-66d9498f5-f84ct   1/1      Running              0           53m
order-service-56cf5dc64b-j6fkq      0/1      ImagePullBackOff    0           75m
order-service-56cf5dc64b-qtcsj      0/1      ImagePullBackOff    0           75m
order-service-56cf5dc64b-xghnn      0/1      ImagePullBackOff    0           75m
postgres-7475d95dcf-ztl9d          1/1      Running              0           75m
ubuntu@ip-10-0-2-77:~$
```

Figura 3.6: Verifica dello stato dei Pod sul Nodo Admin AWS

- **Navigazione:** Terminata la fase di deploy su entrambi i nodi, l'accesso alla piattaforma avviene dal browser tramite l'URL fornito dall'Application Load Balancer. L'ALB si occuperà di smistare automaticamente il traffico basandosi sul *path* dell'URL.



Example of deploying an affordable e-commerce platform: Scalamarket

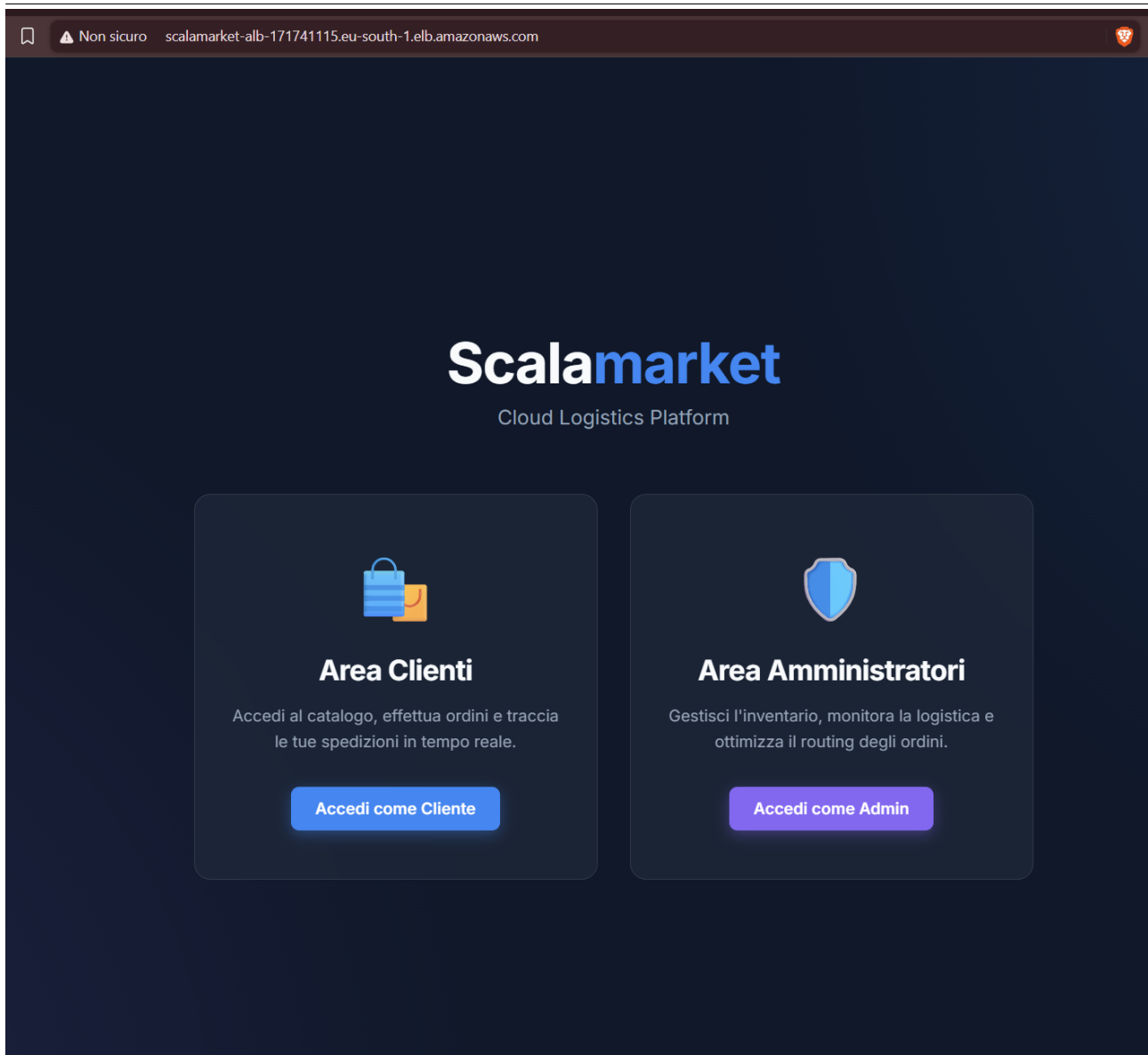


Figura 3.7: Architettura deployata e accessibile tramite Application Load Balancer AWS



Capitolo 4

Proposte di Integrazione e Sviluppo Futuro

Questo capitolo presenta le proposte per l'evoluzione dell'architettura e per l'esecuzione di test prestazionali atti a validare le scelte implementative.

4.1 Integrazioni Edge Computing e Scaling On-Premise

L'architettura a microservizi basata su K3s (una versione leggera di Kubernetes) è intrinsecamente flessibile, rendendosi adatta non solo al Cloud ma anche ad ambienti ibridi o puramente locali, consentendo l'integrazione con dispositivi IoT periferici.

Scenario A: Scaling in un Cluster On-Premise

Invece di fare affidamento unicamente su istanze Cloud come su AWS, è possibile sfruttare la natura distribuita di Kubernetes per scalare orizzontalmente all'interno di un **cluster fisico locale** (On-Premise).

- Un PC desktop mediamente potente può essere designato come **Master Node** (Control Plane). Si possono poi collegare alla stessa rete LAN 2 o 3 portatili o macchine meno performanti che fungeranno da **Worker Node**.
- **Vantaggi di scalabilità e resilienza:** Questo approccio permette di aggiungere o rimuovere potenza di calcolo (scalando orizzontalmente i Worker) a seconda delle necessità. In caso di guasto di una macchina, è possibile osservare in tempo reale come Kubernetes intervenga per riavviare i Pod (i microservizi) sui nodi superstiti, mantenendo il servizio pienamente operativo.



Scenario B: Integrazione di dispositivi Edge Computing (ESP32 / ESP8266)

Per migliorare ulteriormente il progetto Scalamarket connettendolo al mondo reale, si propone l'integrazione di microcontrollori economici come l'ESP32 o l'ESP8266. Pur non potendo eseguire internamente Docker o Kubernetes per via delle limitate risorse hardware, queste schede sono perfette per agire da **IoT Edge Devices**. Possono raccogliere dati fisici e interagire direttamente con i microservizi esposti dal cluster:

- **Lettore di codici a barre Smart (Warehouse):** Un ESP32 collegato a un modulo ottico scanner può essere utilizzato dagli operatori in magazzino. Alla scansione di una merce, il dispositivo edge invia una richiesta HTTP POST via Wi-Fi direttamente all'endpoint `/api/inventory/`, aggiornando le scorte nel database in tempo reale senza dover passare per un'interfaccia web o macchine intermedie.
- **Monitoraggio Flotta (Veicoli):** Un ESP8266 dotato di modulo GPS, installato sui mezzi logistici, può eseguire chiamate REST periodiche verso `l'order-service`. Questo consente il tracciamento dinamico della posizione del mezzo e l'aggiornamento automatico dello stato della consegna, offrendo un tracking preciso ai clienti.

4.2 Benchmark Prestazionali: Cloud vs Locale

Per valutare in maniera oggettiva l'efficacia del deployment e comprendere in quali scenari una soluzione in locale (On-Premise) possa risultare più conveniente e performante rispetto a una soluzione Cloud su AWS, si propone l'utilizzo di strumenti di load testing come **Locust** o **Apache Bench (ab)**.

Eseguire i seguenti test in entrambi gli ambienti (AWS e On-Premise locale) consentirà di generare metriche comparative affidabili:

Test 1: Sfogliare il Catalogo (Read-Intensive)

- **Azione:** Generare un volume massivo di chiamate GET `/api/inventory/` per richiedere la lista dei prodotti.
- **Obiettivo Comparativo:** Misurare le **RPS (Richieste per Secondo)** massime gestibili mantenendo una latenza inferiore ai 200 millisecondi. Risulterà evidente se l'overhead di rete del Cloud



Example of deploying an affordable e-commerce platform: Scalamarket

giustifichi i costi rispetto alle alte prestazioni a latenza zero di una rete LAN locale.

Test 2: Inserimento Ordini (Write-Intensive e Logica)

- **Azione:** Simulazione di un checkout di massa (es. Black Friday) inviando contemporaneamente centinaia di richieste POST `/api/orders/`.
- **Obiettivo Comparativo:** Analizzare la reattività del database e la corretta esecuzione della logica condizionale. Questo benchmark confermerà se l'architettura su nodi Cloud *general purpose* riesce a superare un server locale ottimizzato per scritture (ad esempio dotato di SSD NVMe). Servirà anche a verificare l'efficacia della replicazione orizzontale a 3 Pod.

Test 3: Spike Test (Picco Improvviso di traffico)

- **Azione:** Incremento istantaneo degli utenti concorrenti (da 10 a 1000 in una frazione di secondo) su tutte le API.
- **Obiettivo Comparativo:** Testare i limiti dell'Ingress Controller (NGINX in locale vs Traefik/ALB su AWS) e la sua capacità di resistere ai colli di bottiglia, restituendo codici di errore (502/503) per proteggere i Pod da esaurimento memoria (OOM), o scalando tempestivamente se integrato con metriche HPA.



Capitolo 5

Documentazione Endpoints API

Questo documento presenta tutti gli endpoint implementati nell'applicativo Scalamarket. Il sistema è basato su un'architettura a microservizi, composta da: Auth Service (Porta 5001), Inventory Service (Porta 5002) e Order Service (Porta 5003). Di seguito, le tabelle sono suddivise in base ai file di routing (es. `client.py` e `admin.py`) che implementano i vari endpoint.

Auth Service

/client/register			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Registra un nuovo utente cliente nel sistema.	{username, password}	201 (Created) 400 (Bad Request)
/client/login			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Autentica un cliente e ne restituisce le informazioni.	{username, password}	200 (OK) 401 (Unauthorized)

Tabella 5.1: Auth Service - `routes/client.py`



Example of deploying an affordable e-commerce platform: Scalamarket

/admin/register			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Registra un nuovo amministratore o gestore filiale.	{username, password, role, location_id}	201 (Created) 400 (Bad Request)
/admin/login			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Autentica un amministratore e ne restituisce i dati.	{username, password}	200 (OK) 401 (Unauthorized)
/admin/users			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Restituisce la lista di tutti gli utenti registrati.	Nessuno	200 (OK)
/admin/users/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
DELETE	Elimina definitivamente l'utente specificato dall'ID.	Nessuno (id nell'URL)	200 (OK) 404 (Not Found)

Tabella 5.2: Auth Service - routes/admin.py



Inventory Service

/client/catalog			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Ottiene il catalogo completo degli articoli disponibili per i clienti.	Nessuno	200 (OK)

Tabella 5.3: Inventory Service - `routes/client.py`



Example of deploying an affordable e-commerce platform: Scalamarket

/admin/article			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Aggiunge un nuovo articolo all'inventario con relative giacenze.	{name, description, price, quantities}	201 (Created) 400 (Bad Request)
/admin/inventory			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Visualizza lo stato globale di inventario e giacenze.	Nessuno	200 (OK)
/admin/inventory			
	Descrizione	Parametri Richiesti	Codici HTTP
PUT	Aggiorna la quantità di giacenza di un articolo in una specifica filiale.	{article_id, location_id, quantity}	200 (OK) 400 (Bad Request)
/admin/article/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
DELETE	Rimuove definitivamente un articolo dal catalogo e tutte le sue giacenze.	Nessuno (id nell'URL)	200 (OK) 404 (Not Found)
/admin/inventory/<article_id>/<location_id>			
	Descrizione	Parametri Richiesti	Codici HTTP
DELETE	Rimuove una specifica voce di inventario per un articolo in una filiale.	Nessuno (nell'URL)	200 (OK) 404 (Not Found)
/admin/internal/allocate			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Effettua l'allocazione logistica prelevando gli articoli dalle filiali.	{order_id, items}	200 (OK) 400 (Bad Request)

Tabella 5.4: Inventory Service - routes/admin.py



Order Service

/client/order			
	Descrizione	Parametri Richiesti	Codici HTTP
POST	Crea e piazza un nuovo ordine a nome dell'utente specificato.	{user_id, items}	201 (Created) 400 (Bad Request)
/client/order/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Traccia lo stato di uno specifico ordine in base all'ID fornito.	Nessuno (id nell'URL)	200 (OK) 404 (Not Found)
/client/order/user/<id>			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Ottiene lo storico di tutti gli ordini completati e falliti dell'utente.	Nessuno (id nell'URL)	200 (OK)

Tabella 5.5: Order Service - routes/client.py

/admin/orders			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Restituisce tutti gli ordini registrati nel sistema.	Nessuno	200 (OK)
/admin/logistics			
	Descrizione	Parametri Richiesti	Codici HTTP
GET	Visualizza l'uso corrente e totale dei veicoli logistici per le filiali.	Nessuno	200 (OK)
/admin/logistics			
	Descrizione	Parametri Richiesti	Codici HTTP
PUT	Aggiorna i veicoli logistici totali di una determinata filiale.	{location_id, total_vehicles}	200 (OK) 404 (Not Found)

Tabella 5.6: Order Service - routes/admin.py



Capitolo 6

Interfaccia Utente (UI/UX)

6.1 Design e Suddivisione dei Ruoli

L'interfaccia utente di Scalamarket separa l'esperienza dell'utente finale (il Cliente) da quella degli operatori di back-office (gli Amministratori/Gestori Filiera), mantenendo un'architettura frontend unificata. Il design adotta uno stile moderno, caratterizzato da temi scuri (Dark Mode), palette di colori vivaci e layout responsivo.

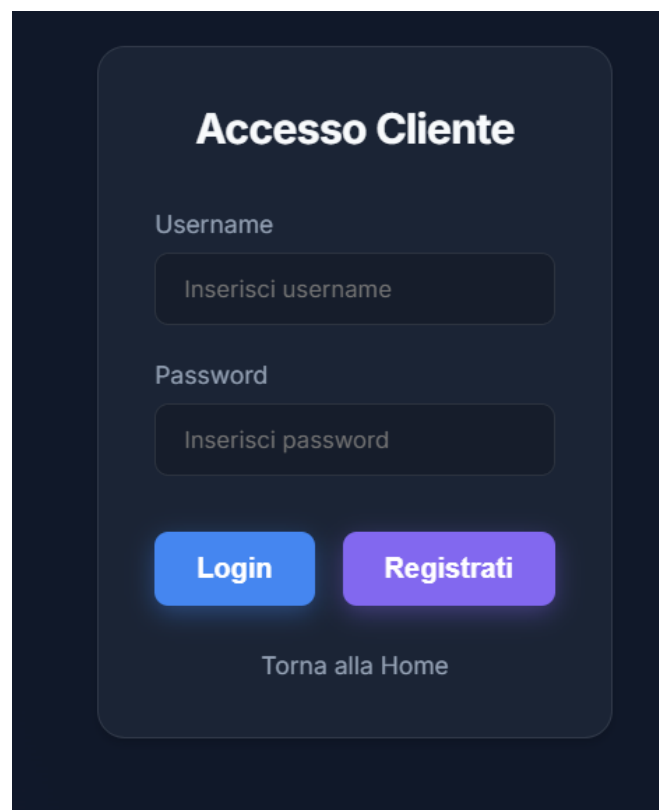


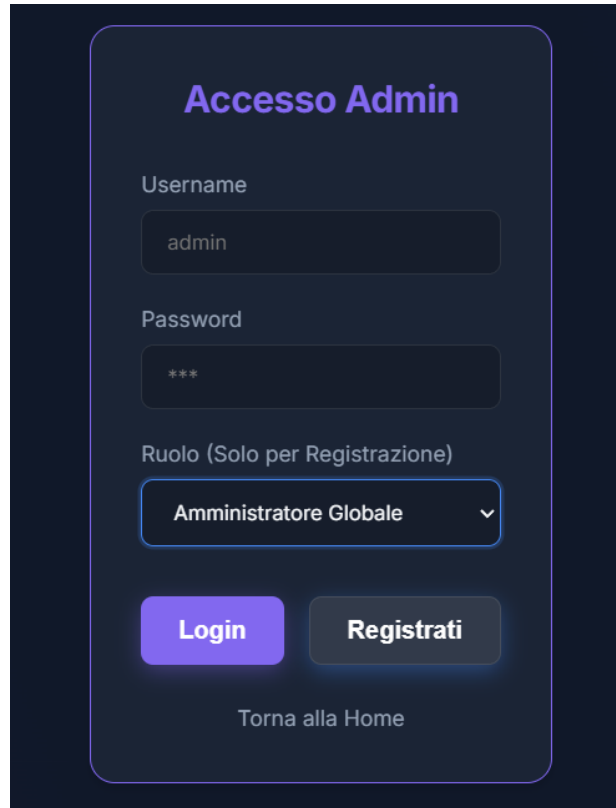
Figura 6.1: Schermata di Accesso e Registrazione (Area Clienti)

L'area di back-office, invece, offre un sistema di registrazione basato su ruoli: è possibile iscriversi come Amministratore Globale, per una supervisione totale della piattaforma, oppure come Gestore Filiale, con la



Example of deploying an affordable e-commerce platform: Scalamarket

facoltà di selezionare una delle tre filiali operative disponibili al momento dell'iscrizione.



The image shows a dark-themed user interface for an 'Accesso Admin' (Admin Access) form. The form is centered on a dark background and contains the following elements:

- Accesso Admin**: The title of the form, displayed in a light blue font.
- Username**: A label above a text input field containing the text 'admin'.
- Password**: A label above a password input field containing three asterisks '***'.
- Ruolo (Solo per Registrazione)**: A label above a dropdown menu.
- Amministratore Globale**: The selected option in the dropdown menu, with a downward arrow icon.
- Login**: A blue button with white text.
- Registrati**: A grey button with white text.
- Torna alla Home**: A link or button at the bottom of the form.

Figura 6.2: Registrazione di un nuovo Amministratore Globale



Example of deploying an affordable e-commerce platform: Scalamarket

Figura 6.3: Registrazione di un Gestore Filiale con scelta della sede

Si precisa che gli attuali moduli di autenticazione e registrazione, sebbene pienamente funzionali ai fini dell'illustrazione delle dinamiche di accesso basate sui ruoli (RBAC), sono stati concepiti come *Proof of Concept* (PoC) per dimostrare le capacità architetturali e di routing della piattaforma. Tali schermate e le relative implementazioni lato client risultano pertanto semplificate; per un effettivo rilascio in ambiente di produzione (*enterprise*), si renderebbe necessaria l'integrazione di strati di sicurezza più avanzati, policy di password complesse, validazione multi-fattore (MFA) e sistemi di Identity Access Management (IAM) dedicati.

6.2 Pannello Cliente

La dashboard cliente mostra il catalogo degli articoli, con disponibilità residua globale calcolata dinamicamente interrogando il servizio inventario. Il carrello impedisce l'aggiunta di quantità superiori alla disponibilità.



Example of deploying an affordable e-commerce platform: Scalamarket

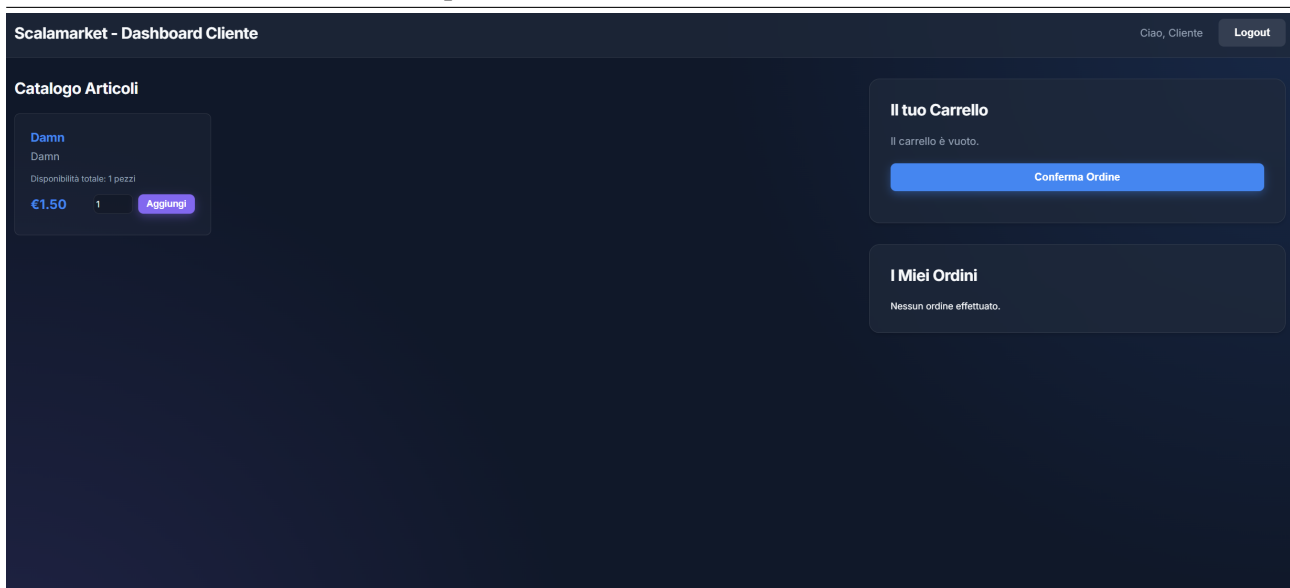


Figura 6.4: Dashboard Cliente: Catalogo Prodotti e Carrello



Figura 6.5: Conferma dell'ordine andato a buon fine (Area Cliente)

6.3 Pannelli di Amministrazione

Accedendo alla dashboard di amministrazione, gli operatori e i gestori di filiera hanno a disposizione sezioni specifiche per l'aggiunta di nuovi articoli, la gestione logistica (aggiornamento dei mezzi di trasporto) e l'inventario globale consolidato.



Example of deploying an affordable e-commerce platform: Scalamarque

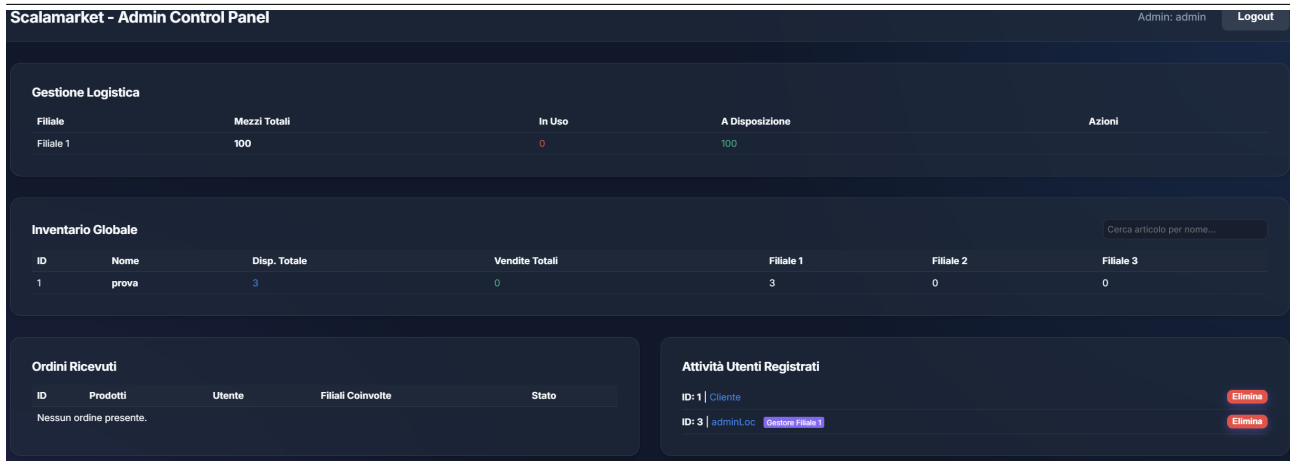


Figura 6.6: Dashboard Amministratore (Inventario, Logistica e Analytics)

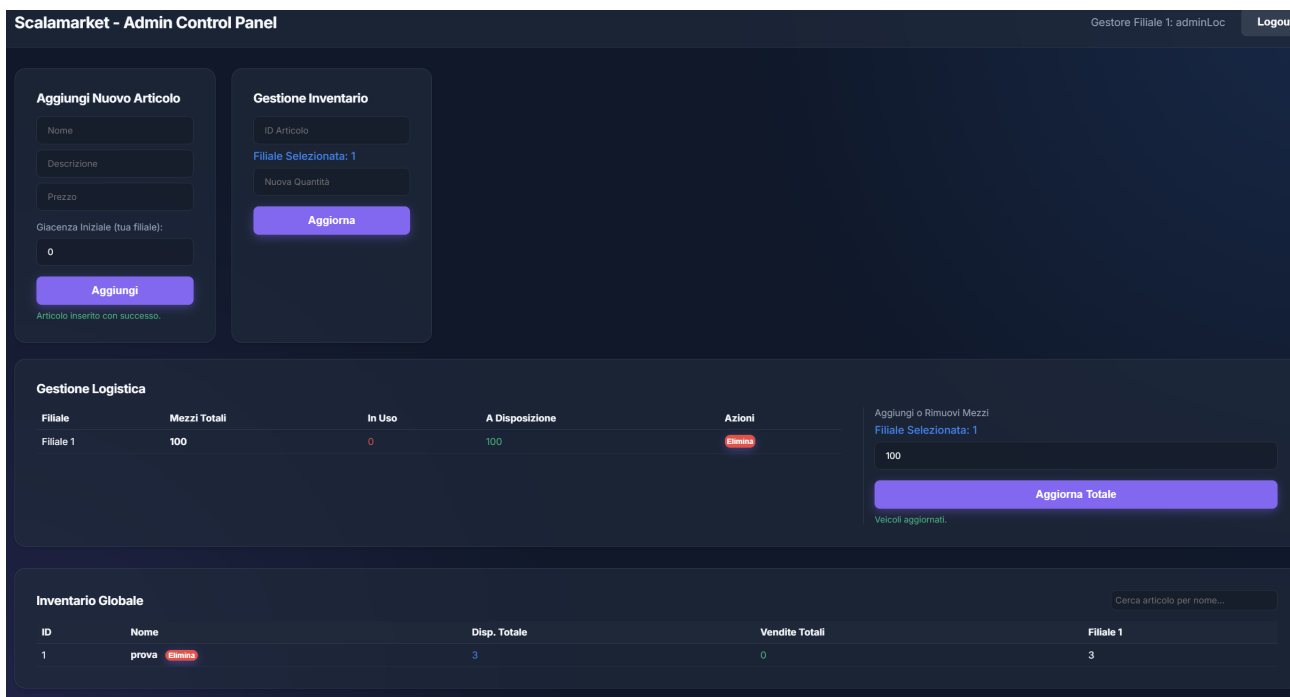


Figura 6.7: Dettaglio della gestione locale della singola filiale (Gestore Filiera)



Example of deploying an affordable e-commerce platform: Scalamarket

Scalamarket - Admin Control Panel

Admin: adminLogout

Gestione Logistica

Filiale	Mezzi Totali	In Uso	A Disposizione	Azioni
Filiale 1	100	1	99	

Inventario Globale

Cerca articolo per nome...

ID	Nome	Disp. Totale	Vendite Totali	Filiale 1	Filiale 2	Filiale 3
1	prova	0	3	0	0	0

Ordini Ricevuti

ID	Prodotti	Utente	Filiali Coinvolte	Stato
#1	prova (x3)	Cliente	1	ROUTED

Attività Utenti Registrati

ID: 1 | Cliente

ID: 3 | adminLoc

Gestione Filiale 1

Figura 6.8: Aggiornamento in tempo reale delle risorse a seguito di un ordine

Scalamarket - Admin Control Panel

Admin: adminLogout

Gestione Logistica

Filiale	Mezzi Totali	In Use	A Disposizione	Azioni
Filiale 1	100	1	99	

Inventario Globale

Cerca articolo per nome...

ID	Nome	Disp. Totale	Vendite Totali	Filiale 1	Filiale 2	Filiale 3
1	prova	0	3	0	0	0

Ordini Ricevuti

ID	Prodotti	Utente	Filiali Coinvolte	Stato
#1	prova (x3)	Cliente	1	ROUTED

Attività Utenti Registrati

ID: 1 | Cliente

Elimina

ID: 3 | adminLoc

Gestione Filiale 1

Elimina

Figura 6.9: Visualizzazione della ricezione dell'ordine dal pannello Gestore Filiale



Capitolo 7

Guide al Deployment

7.1 Deployment in Ambiente Locale

Fase 1 – Avvio Barebone

```
1 # Build immagini Docker (dalla root del progetto)
2 docker build -t cpiac_frontend:v6 -f frontend/Dockerfile
  frontend/
3 docker build -t cpiac_auth:v6      -f microservices/auth_service
  /Dockerfile .
4 docker build -t cpiac_inventory:v6 -f microservices/
  inventory_service/Dockerfile .
5 docker build -t cpiac_order:v6     -f microservices/
  order_service/Dockerfile .
6
7 # Deploy di tutte le risorse Kubernetes
8 kubectl apply -f k8s/
9
10 # Port-forward per accesso locale (alternativa all'Ingress)
11 kubectl port-forward svc/auth-service      5001:5001 &
12 kubectl port-forward svc/inventory-service 5002:5002 &
13 kubectl port-forward svc/order-service    5003:5003 &
14
15 kubectl get pods # Verifica stato
```

Listing 7.1: Avvio infrastruttura Barebone

Fase 2 – Avvio con Ridondanza e Ingress

```
1 # (Una-tantum) Installazione NGINX Ingress Controller
2 kubectl apply -f https://raw.githubusercontent.com/kubernetes/
  ingress-nginx/\
3 controller-v1.10.0/deploy/static/provider/cloud/deploy.yaml
4
5 # Deploy delle risorse base (escludendo ingress.yaml di default
  per AWS)
6 kubectl apply -f k8s/postgres-pv-pvc.yaml
7 kubectl apply -f k8s/postgres-configmap.yaml
```



Example of deploying an affordable e-commerce platform: Scalamarket

```
8 kubectl apply -f k8s/postgres-deployment.yaml
9 kubectl apply -f k8s/postgres-service.yaml
10 kubectl apply -f k8s/frontend-deployment.yaml
11 kubectl apply -f k8s/frontend-service.yaml
12 kubectl apply -f k8s/auth-deployment.yaml
13 kubectl apply -f k8s/auth-service.yaml
14 kubectl apply -f k8s/inventory-deployment.yaml
15 kubectl apply -f k8s/inventory-service.yaml
16 kubectl apply -f k8s/order-deployment.yaml
17 kubectl apply -f k8s/order-service.yaml
18
19 # Applicazione dell'Ingress specifico locale compatibile con
    NGINX
20 kubectl apply -f k8s/ingress-local.yaml
21
22 # (Opzionale) Scalare i pod a 1 replica in caso di limitazioni
    CPU in locale
23 kubectl scale deployment frontend auth-service inventory-
    service order-service --replicas=1
24
25 # Verifica repliche e stato Ingress
26 kubectl get pods
27 kubectl get ingress
28
29 # Accesso via browser: Naviga utilizzando l'indirizzo esposto
    dall'Ingress (localhost o l'IP ottenuto)
```

Listing 7.2: Avvio infrastruttura con repliche e Ingress

Per azzerare e ricreare l'intera infrastruttura da zero (ad esempio dopo una modifica strutturale al database):

```
1 kubectl delete -f k8s/
2 sleep 5
3 kubectl apply -f k8s/
```

Listing 7.3: Reset completo del cluster

7.2 Deployment in AWS tramite Infrastructure-as-Code (Terraform)

Configurazione IAM per Terraform:

Per il corretto provisioning, è richiesto un utente programmatico in AWS IAM



Example of deploying an affordable e-commerce platform: Scalamarket

Durante la generazione delle *Access Key*, selezionare lo Use Case **Command Line Interface (CLI)**, poiché Terraform interagisce con le API di AWS in modalità terminale.

I passi per eseguire lo script sono i seguenti:

1. **Generazione Chiave SSH:** Terraform richiede l'esistenza di una chiave SSH locale per l'autenticazione remota sui server. Se non si possiede già una chiave, generarne una eseguendo da terminale il comando:
`ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa.`
2. **Configurazione Credenziali AWS:** Eseguire `aws configure` per configurare la CLI con le *Access Keys* ottenute.
3. **Inizializzazione:** Eseguire `terraform init`.
4. **Verifica e Applicazione:** Eseguire `terraform plan` e infine `terraform apply`.

Accesso Remoto e Deploy Applicativo:

Al termine dell'esecuzione di Terraform, verranno stampati a video il DNS name dell'ALB e gli IP Pubblici delle due istanze EC2. L'infrastruttura a questo punto è in esecuzione, ma le applicazioni devono ancora essere deployate. I passaggi sono:

- **Accesso SSH:** Collegarsi in SSH alle macchine tramite i seguenti comandi (basati sull'output di Terraform):

```
– Nodo      Clienti:      ssh -i ~/.ssh/id_rsa
                        ubuntu@<IP_CLIENTI>
– Nodo      Admin:        ssh -i ~/.ssh/id_rsa
                        ubuntu@<IP_ADMIN>
```

- **Clone Selettivo (Sparse-Checkout):** Per evitare di scaricare file inutili ai fini del deploy (come questa documentazione o i file Terraform), utilizzare i seguenti comandi su entrambi i nodi:

```
git clone --no-checkout <TUO_URL_GIT>
cd CPIAC_Scalamarket
git sparse-checkout init --cone
git sparse-checkout set frontend init-scripts microservices
git checkout main
```

- **Deploy K8s - Nodo Clienti:** (Assicurarsi di essere nella cartella `CPIAC_Scalamarket`). Poiché su AWS i servizi sono distribuiti su



Example of deploying an affordable e-commerce platform: Scalamarket

due nodi indipendenti, il servizio ordini ha bisogno di conoscere l'indirizzo pubblico del bilanciatore per comunicare con l'inventario (mentre in un ambiente -LOCAL comunicherebbero tramite DNS interno). Sostituendo URL_ALB con l'indirizzo del tuo Load Balancer, esegui:

```
sed -i "s|value: \"http://inventory-service:5002\"|v

sudo docker build -t cpiac_frontend:v6 -f frontend/D
sudo docker build -t cpiac_auth:v6      -f microservic
sudo docker build -t cpiac_order:v6     -f microservic

sudo docker save cpiac_frontend:v6 cpiac_auth:v6 cpi
sudo k3s ctr images import clients_images.tar

sudo kubectl apply -f k8s/postgres-pv-pvc.yaml
sudo kubectl apply -f k8s/postgres-configmap.yaml
sudo kubectl apply -f k8s/postgres-deployment.yaml
sudo kubectl apply -f k8s/postgres-service.yaml

sudo kubectl apply -f k8s/frontend-deployment.yaml
sudo kubectl apply -f k8s/frontend-service.yaml
sudo kubectl apply -f k8s/auth-deployment.yaml
sudo kubectl apply -f k8s/auth-service.yaml
sudo kubectl apply -f k8s/order-deployment.yaml
sudo kubectl apply -f k8s/order-service.yaml
sudo kubectl apply -f k8s/ingress.yaml
```

- **Deploy K8s - Nodo Admin:**

```
sudo docker build -t cpiac_inventory:v6 -f microserv

sudo docker save cpiac_inventory:v6 > admin_images.t
sudo k3s ctr images import admin_images.tar

sudo kubectl apply -f k8s/postgres-pv-pvc.yaml
sudo kubectl apply -f k8s/postgres-configmap.yaml
sudo kubectl apply -f k8s/postgres-deployment.yaml
sudo kubectl apply -f k8s/postgres-service.yaml
```



Example of deploying an affordable e-commerce platform: Scalamarket

```
sudo kubectl apply -f k8s/inventory-deployment.yaml
sudo kubectl apply -f k8s/inventory-service.yaml
sudo kubectl apply -f k8s/ingress.yaml
```

- **Navigazione:** Accedere dal browser non tramite gli IP delle macchine, ma tramite l'URL fornito dall'Application Load Balancer (es. `http://<URL_ALB>`). L'ALB smisterà automaticamente il traffico basandosi sul *path* dell'URL.